МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01. «Математика и компьютерные науки»

Отчет о выполнении научно-исследовательской работы На тему: «Разработка имитатора пользовательских заявок и анализ эффективности распределения вычислительных ресурсов в планировщике SLURM»

Выполнил студент
группы 5130201/40002 _____ Храмов В.Д.

Принял старший преподаватель _____ Чуватов М.В.

«___» _____ 2026 г.

Санкт-Петербург
2026

Реферат

Отчет по научно-исследовательской работе изложен на ? страницах, содержит ? иллюстраций, ? таблицы, ? использованных источника, ? приложения.

Ключевые слова: распределенные вычислительные системы, планировщик задач, slurm, имитационное моделирование, нормальное распределение, логнормальное распределение, анализ датасета, кластер, ssh.

Объектом исследования являются методы диспетчеризации и планирования вычислительных ресурсов в распределенных кластерных системах под управлением СУПО SLURM на базе операционной системы Linux в среде виртуализации.

Целью работы является разработка программного имитатора пользовательских заявок на основе статистического анализа исторических данных суперкомпьютера (вариант: UID 50430, JobName run_bwa.sh) и исследование динамики прохождения очередей задач в реальном планировщике SLURM с использованием механизмов масштабирования времени.

Результатом разработки стал комплекс программно-управляемых скриптов на языках Python и Bash, обеспечивающий фильтрацию исходного датасета acct_0921-0923, вычисление статистических параметров человеческой ошибки ($\mu = 3.8621$, $\sigma = 0.8489$), генерацию случайного потока задач с усечением границ и их автоматическую отправку в пакетный диспетчер. На базе развернутого 5-узлового виртуального кластера проведено экспериментальное тестирование системы, в ходе которого средствами SLURM была собрана и проанализирована итоговая статистика соотношения «план – факт» для верификации математической модели.

Содержание

Термины и определения4Перечень сокращений и обозначений6Введение7

1 ОРГАНИЗАЦИЯ ВИРТУАЛЬНОЙ ВЫЧИСЛИТЕЛЬНОЙ СРЕДЫ 8
1.1 Выбор и развёртывание среды виртуализации 8
1.2 Архитектура виртуального кластера 9
1.3 Решение проблемы совместимости с архитектурой ARM . . . 9
1.4 Настройка виртуальной сети и разрешения имён 10
1.5 Настройка SSH с авторизацией по ключам 10
1.6 Установка и настройка munge 11
1.7 Установка и настройка SLURM 11
1.8 Клонирование вычислительных узлов 12

2 РАЗРАБОТКА ИМИТАТОРА ПОЛЬЗОВАТЕЛЬСКИХ ЗАЯВОК 12
2.1 Анализ исходного набора данных 12
2.2 Предварительный анализ и масштабирование данных 12
2.3 Выбор модели ошибки оценки времени выполнения 13
2.4 Вычисление статистических параметров 13
2.5 Алгоритм генерации пользовательской заявки 13
2.6 Реализация программы-имитатора 14
2.7 Сбор статистики поставленных в очередь заявок 15

3 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ И АНАЛИЗ ДАННЫХ 15
3.1 Условия проведения эксперимента 15
3.2 Результаты постановки заявок в очередь 15
3.3 Анализ отклонений «план – факт» 16
3.4 Выводы по результатам экспериментов 16

Список литературы 19

Приложение А. Исходный код имитатора 20

Термины и определения

1. VirtualBox (Oracle VM VirtualBox) — программный продукт виртуализации для операционных систем Windows, Linux, FreeBSD, macOS, Solaris/OpenSolaris и других, позволяющий запускать гостевые операционные системы в изолированной среде на базе хост-системы.
2. Виртуализация — технология создания программного (виртуального) представления вычислительных ресурсов (процессора, памяти, накопителей, сетевых интерфейсов), позволяющая запускать несколько независимых операционных систем на одном физическом устройстве.
3. Виртуальная машина (ВМ) — программная эмуляция вычислительного устройства, предоставляющая гостевой операционной системе доступ к выделенным ей виртуальным аппаратным ресурсам.
4. Гипервизор — программный или программно-аппаратный компонент, обеспечивающий одновременное выполнение нескольких операционных систем на одном физическом компьютере и управляющий распределением ресурсов между ними.
5. VirtIO SCSI — паравиртуализированный драйвер контроллера накопителей, обеспечивающий взаимодействие гостевой операционной системы с виртуальным диском через оптимизированный интерфейс гипервизора, применяемый, в частности, для обеспечения совместимости при эмуляции процессорной архитектуры, отличной от архитектуры хост-системы.
6. SLURM (Simple Linux Utility for Resource Management) — система управления заданиями и ресурсами вычислительного кластера с открытым исходным кодом, обеспечивающая постановку заданий в очередь, их планирование и распределение по вычислительным узлам.
7. Управляющий узел (head node, control node) — узел вычислительного кластера, на котором выполняются диспетчер SLURM и средства управления кластером, отвечающие за приём, планирование и распределение заданий по вычислительным узлам.
8. Вычислительный узел (compute node) — узел вычислительного кластера, предоставляющий ресурсы (процессорное время, память) для непосредственного выполнения заданий, поставленных в очередь диспетчера.

9. SSH (Secure Shell) — сетевой протокол прикладного уровня, позволяющий производить удалённое управление операционной системой и туннелирование TCP-соединений, в том числе с использованием авторизации по криптографическим ключам без передачи пароля по сети.
10. Munge — служба аутентификации, используемая SLURM для проверки подлинности сообщений, которыми обмениваются узлы кластера, на основе общего секретного ключа.
11. Разрешение имён — процесс сопоставления символьных имён узлов сети соответствующим им IP-адресам, выполняемый средствами DNS или локального файла соответствий (/etc/hosts).
12. Заявка (задание, job) — описание вычислительной задачи, поставленной пользователем в очередь диспетчера кластера, включающее запрошенные ресурсы и оценку времени выполнения.
13. Очередь заданий диспетчера — упорядоченный список заявок, принятых диспетчером SLURM и ожидающих выделения ресурсов для запуска на вычислительных узлах.
14. Нормальное распределение — непрерывное распределение вероятностей случайной величины, полностью определяемое математическим ожиданием и дисперсией, график плотности которого имеет симметричную колоколообразную форму.
15. Логнормальное распределение — распределение вероятностей случайной величины, логарифм которой имеет нормальное распределение с параметрами μ и σ.
16. Среднее арифметическое — статистическая характеристика выборки, равная сумме всех её значений, делённой на их количество.
17. Стандартное отклонение — статистическая характеристика разброса значений случайной величины относительно её среднего арифметического, равная квадратному корню из дисперсии.

Перечень сокращений и обозначений

ВМ — виртуальная машина

ОЗУ — оперативное запоминающее устройство

ОС — операционная система

ПО — программное обеспечение

ЦП — центральный процессор

CPU (Central Processing Unit) — центральный процессор

GID (Group Identifier) — идентификатор группы пользователей

HPC (High Performance Computing) — высокопроизводительные вычисления

IP (Internet Protocol) — межсетевой протокол

NAT (Network Address Translation) — преобразование сетевых адресов

RAM (Random Access Memory) — оперативная память

SLURM (Simple Linux Utility for Resource Management) — система управления заданиями и ресурсами вычислительного кластера

SSH (Secure Shell) — сетевой протокол прикладного уровня для удалённого управления операционной системой

TCP (Transmission Control Protocol) — протокол управления передачей

UID (User Identifier) — идентификатор пользователя

Введение

В современном мире, где объёмы обрабатываемых данных постоянно растут, а вычислительные задачи становятся всё сложнее, технологии высокопроизводительных вычислений (HPC) приобретают критически важное значение. Их применение позволяет объединять ресурсы множества вычислительных узлов в единый кластер, обеспечивая тем самым существенное увеличение производительности и сокращение времени решения сложных научных и инженерных задач.

Эффективная эксплуатация вычислительного кластера невозможна без системы управления заданиями и ресурсами, обеспечивающей приём заявок пользователей, их постановку в очередь и распределение по вычислительным узлам в соответствии с заданной политикой планирования. Одной из наиболее распространённых систем такого класса является SLURM (Simple Linux Utility for Resource Management), широко используемая как в коммерческих, так и в научно-исследовательских вычислительных центрах.

Освоение принципов администрирования и эксплуатации вычислительного кластера на базе SLURM не требует обязательного наличия физического кластера: для учебных и исследовательских целей вычислительная среда может быть воспроизведена в виде набора виртуальных машин. Такие инструменты, как VirtualBox, позволяют создавать изолированные виртуальные среды, в которых каждая машина выполняет роль отдельного узла кластера, что делает возможным изучение архитектуры распределённой системы средствами одного физического компьютера.

Операционные системы семейства Linux, в частности дистрибутив Fedora Server, благодаря своей гибкости, открытости и широкому набору системных утилит, предоставляют все необходимые инструменты для организации управляющего и вычислительных узлов кластера в режиме интерфейса командной строки. Для организации безопасного взаимодействия между узлами кластера и удалённого администрирования незаменим протокол SSH (Secure Shell), обеспечивающий авторизацию по криптографическим ключам без передачи пароля по сети, а также служба munge, отвечающая за аутентификацию сообщений между демонами SLURM.

Помимо развёртывания и настройки самой вычислительной среды,

важной задачей является имитация реальной пользовательской нагрузки на кластер с целью проверки корректности работы диспетчера и последующего анализа эффективности планирования заданий. Для построения такого имитатора целесообразно опираться не на произвольные, а на статистически обоснованные параметры заявок, полученные путём анализа реальных данных учёта выполненных задач (accounting data) действующего вычислительного кластера. Применение математического аппарата теории вероятностей и математической статистики, в частности нормального и логнормального распределений, позволяет генерировать синтетические заявки, по своим временным характеристикам приближенные к наблюдаемым в реальных системах.

В ходе настоящей НИР рассматриваются технологии организации вычислительного кластера на базе виртуализации и Linux, включая развёртывание управляющего и вычислительных узлов средствами VirtualBox, настройку сетевого взаимодействия и SSH-доступа, установку и конфигурирование диспетчера SLURM, а также разработку имитатора пользовательских заявок, основанного на статистическом анализе набора учётных данных кластера.

1 ОРГАНИЗАЦИЯ ВИРТУАЛЬНОЙ ВЫЧИСЛИТЕЛЬНОЙ СРЕДЫ

1.1 Выбор и развёртывание среды виртуализации

В качестве среды виртуализации в данной работе использовался программный продукт Oracle VM VirtualBox, выбор которого обусловлен его свободным распространением, поддержкой широкого спектра гостевых операционных систем и наличием официальной сборки для архитектуры ARM64, что является необходимым условием для развёртывания виртуальной среды на хост-системе с процессором Apple Silicon (MacBook Air, чип Apple M2, 8 ГБ ОЗУ).

В соответствии с вариантом задания в качестве гостевой операционной системы был выбран дистрибутив Fedora Server в редакции для архитектуры aarch64. Установка и функционирование системы осуществлялись в режиме интерфейса командной строки, без графической оболочки.

1.2 Архитектура виртуального кластера

Для имитации вычислительных ресурсов суперкомпьютера было развёрнуто пять виртуальных машин: один управляющий узел (control node), на котором размещаются диспетчер SLURM и средства разработки, и четыре вычислительных узла (compute node), предоставляющие ресурсы для выполнения заданий. Конфигурация виртуальных машин, выбранная с учётом ограничений хост-системы по объёму оперативной памяти, приведена в таблице 1.

Таблица 1: Конфигурация узлов виртуального кластера

Узел	Роль	ЦП (ядра)	ОЗУ (МБ)
control	управляющий узел, slurmctld	2	2048
node1	вычислительный узел, slurmd	1	1024
node2	вычислительный узел, slurmd	1	1024
node3	вычислительный узел, slurmd	1	1024
node4	вычислительный узел, slurmd	1	1024

На каждом узле создана единая непривилегированная учётная запись пользователя, используемая для запуска заданий и работы с диспетчером SLURM; учётная запись root применялась исключительно для установки и настройки системного программного обеспечения.

1.3 Решение проблемы совместимости с архитектурой ARM

В процессе установки гостевой операционной системы Fedora Server на виртуальную машину с эмулируемым контроллером накопителей по умолчанию (SATA/AHCI) была выявлена проблема несовместимости: на хост-системе с архитектурой ARM (Apple M2) образ установки не определял подключённый виртуальный диск, что приводило к невозможности завершить установку.

Для устранения данной проблемы тип контроллера накопителя виртуальной машины был изменён с SATA на VirtIO SCSI — паравиртуализированный драйвер, обеспечивающий корректное взаимодействие гостевой системы с диском при эмуляции архитектуры, отличной от архитектуры хост-машины. После применения данного решения установка операционной системы была успешно завершена на всех пяти виртуальных машинах.

1.4 Настройка виртуальной сети и разрешения имён

Для обеспечения взаимодействия между узлами кластера был настроен отдельный виртуальный сетевой адаптер в режиме <режим сети VirtualBox>, обеспечивающий изолированную локальную сеть для всех пяти виртуальных машин. Каждому узлу был присвоен статический IP-адрес в соответствии с таблицей 2.

Таблица 2: Адресация узлов виртуального кластера

Имя узла	IP-адрес
control	192.168.56.10
node1	192.168.56.11
node2	192.168.56.12
node3	192.168.56.13
node4	192.168.56.14

Разрешение имён узлов обеспечено посредством редактирования файла /etc/hosts на каждой виртуальной машине, что позволило обращаться к узлам по символьным именам (control, node1–node4) без использования отдельного DNS-сервера.

```
192.168.56.10   control
192.168.56.11   node1
192.168.56.12   node2
192.168.56.13   node3
192.168.56.14   node4
```

Листинг 1: Фрагмент файла /etc/hosts

1.5 Настройка SSH с авторизацией по ключам

На управляющем узле для непривилегированной учётной записи была сгенерирована пара криптографических ключей (приватный и публичный) с использованием утилиты ssh-keygen. Публичный ключ был скопирован на все вычислительные узлы средствами утилиты ssh-copy-id, что позволило выполнять удалённое подключение и выполнение команд без запроса пароля.

```
ssh-keygen -t ed25519 -C "cluster-key"
ssh-copy-id user@node1
ssh-copy-id user@node2
```

```
ssh-copy-id user@node3
ssh-copy-id user@node4
```

Листинг 2: Генерация и распространение ключей SSH

В конфигурационном файле демона SSH (/etc/ssh/sshd_config) на всех узлах была отключена авторизация по паролю (PasswordAuthentication no), что обеспечивает использование исключительно ключевой авторизации.

1.6 Установка и настройка munge

Для аутентификации сообщений, которыми обмениваются демоны SLURM на разных узлах, на управляющем узле был сгенерирован общий секретный ключ munge.key, который был скопирован на все вычислительные узлы с сохранением единых владельца и прав доступа (пользователь munge). Служба munge была запущена и добавлена в автозагрузку на всех пяти узлах кластера.

```
# На управляющем узле
sudo /usr/sbin/create-munge-key
scp /etc/munge/munge.key user@node1:/tmp/
# На каждом вычислительном узле
sudo mv /tmp/munge.key /etc/munge/munge.key
sudo chown munge: /etc/munge/munge.key
sudo chmod 400 /etc/munge/munge.key
sudo systemctl enable --now munge
```

Листинг 3: Развёртывание ключа munge

1.7 Установка и настройка SLURM

На управляющем узле был установлен и настроен демон диспетчера slurmctld, на вычислительных узлах — демон slurmd. Основные параметры кластера определены в общем для всех узлов конфигурационном файле /etc/slurm/slurm.conf: имя кластера, адрес управляющего узла, список вычислительных узлов с указанием количества процессоров и объёма оперативной памяти, а также определение единственного раздела (partition), включающего все четыре вычислительных узла.

```
ClusterName=virtual_cluster
SlurmctldHost=control
```

```
NodeName=node[1-4] CPUs=1 RealMemory=1024 State=UNKNOWN
PartitionName=debug Nodes=node[1-4] Default=YES MaxTime=INFINITE State=UP
```

Листинг 4: Фрагмент файла slurm.conf

После запуска демонов на всех узлах корректность конфигурации и доступность вычислительных узлов проверялась командами sinfo и scontrol show nodes.

1.8 Клонирование вычислительных узлов

После полной настройки и проверки работоспособности первого вычислительного узла (установка ОС, сеть, SSH, munge, slurmd) данная виртуальная машина использовалась как базовый образ для клонирования оставшихся трёх вычислительных узлов средствами VirtualBox (Clone с режимом Full Clone). После клонирования на каждой копии выполнялись индивидуализация сетевых параметров (смена IP-адреса и имени узла) и перегенерация машинного идентификатора (/etc/machine-id) во избежание конфликтов в локальной сети.

2 РАЗРАБОТКА ИМИТАТОРА ПОЛЬЗОВАТЕЛЬСКИХ ЗАЯВОК

2.1 Анализ исходного набора данных

Исходные данные для построения статистической модели пользовательских заявок представлены в виде набора учётных записей SLURM acct_0921-0923, содержащего сведения о реально выполненных задачах действующего вычислительного кластера. В соответствии с вариантом задания из общего набора данных были отобраны записи, удовлетворяющие условию: UID = 50430 и JobName = "run_bwa.sh".

В результате фильтрации было получено N_{sample} = <N> записей, использованных для дальнейшего статистического анализа.

2.2 Предварительный анализ и масштабирование данных

Анализ отобранного подмножества показал, что фактическое время выполнения задач (ElapsedRaw) принимает значения порядка <порядок

величин в выборке>, что существенно затрудняет непосредственное воспроизведение данных интервалов времени в условиях учебного стенда. В связи с этим было принято решение о масштабировании исходных временных характеристик с понижающим коэффициентом $k = <\text{k}>$, что позволило перейти от исходных величин, измеряемых часами и днями, к величинам порядка десятков-сотен секунд, пригодным для практической демонстрации работы имитатора и диспетчера.

2.3 Выбор модели ошибки оценки времени выполнения

Для характеристики отклонения пользовательской оценки времени выполнения от фактического времени выполнения задачи использовались две альтернативные модели:

- абсолютное отклонение $\Delta T = T_{\text{заявки}} - T_{\text{факт}}$;
- относительное отклонение $\tau = T_{\text{заявки}} / T_{\text{факт}}$.

По результатам анализа отобранной выборки было установлено, что величина ΔT <сопоставима с / превышает> величину $T_{\text{факт}}$, в связи с чем для дальнейшей работы была выбрана модель относительного отклонения τ.

2.4 Вычисление статистических параметров

Для отобранной и масштабированной выборки были вычислены границы диапазона фактического времени выполнения, а также параметры логарифма выбранной величины ошибки оценки. Результаты вычислений приведены в таблице 3.

Таблица 3: Статистические параметры выборки

Параметр	Значение
$T_{\text{факт, min}}$, с	<min>
$T_{\text{факт, max}}$, с	<max>
$\mu\ (\ln \tau)$	<mu>
$\sigma\ (\ln \tau)$	<sigma>

2.5 Алгоритм генерации пользовательской заявки

Генерация каждой синтетической заявки выполняется в два этапа.

На первом этапе фактическое время выполнения задачи $T_{\text{факт}}$ моделируется как случайная величина, подчинённая нормальному распреде-

лению, ограниченному диапазоном $[T_{\min}, T_{\max}]$, полученным в результате анализа исходных данных (таблица 3). Значение $T_{\text{факт}}$ используется в качестве параметра команды sleep в теле формируемого пакетного сценария SLURM.

На втором этапе пользовательская оценка времени выполнения $T_{\text{заявки}}$ вычисляется как:

$$T_{\text{заявки}} = T_{\text{факт}} \cdot e^{X}, \qquad X \sim \mathcal{N}(\mu, \sigma^2),$$

то есть отклонение оценки от фактического времени моделируется логнормальным распределением с параметрами μ и σ, вычисленными на этапе статистического анализа. Полученное значение $T_{\text{заявки}}$ используется как значение параметра –time пакетного сценария.

Количество узлов, запрашиваемых заявкой, выбирается случайным образом из множества $\{1, 2, 3, 4\}$ с использованием <распределение>, независимо от количества узлов, фигурирующего в исходных данных по варианту.

2.6 Реализация программы-имитатора

Имитатор реализован на языке <язык реализации> и принимает в качестве входного параметра количество заявок N (целое положительное число, не превышающее 999). Для каждой из N заявок программа:

1. генерирует значения $T_{\text{факт}}$, $T_{\text{заявки}}$ и количество узлов согласно описанным выше распределениям;
2. формирует пакетный сценарий SLURM с уникальным именем задачи, указанием количества узлов (–nodes), предельного времени выполнения (–time) и командой sleep <$T_{\text{факт}}$> в качестве имитации полезной нагрузки;
3. автоматически ставит сформированную заявку в очередь диспетчера посредством команды sbatch.

```
#!/bin/bash
#SBATCH --job-name=sim_job_%j
#SBATCH --nodes=<N_nodes>
#SBATCH --time=00:00:<T_request>

sleep <T_actual>
```

Листинг 5: Шаблон пакетного сценария, формируемого имитатором

2.7 Сбор статистики поставленных в очередь заявок

Для последующего сравнения плановых и фактических характеристик выполненных заявок сбор статистики осуществлялся средствами SLURM с использованием команды sacct, позволяющей получить сведения о времени постановки заявки в очередь (Submit), запрошенном времени выполнения (TimelimitRaw), фактическом времени начала и завершения выполнения (Start, End) и фактической продолжительности выполнения (ElapsedRaw). Результат выполнения команды сохраняется в текстовый файл для последующего анализа.

```
sacct --starttime=<начало периода> \
      --format=JobID,JobName,Submit,Start,End,ElapsedRaw,TimelimitRaw,NNodes,
    State \
      --parsable2 --noheader > sim_results.csv
```

Листинг 6: Сохранение статистики выполненных заявок в файл

3 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ И АНАЛИЗ ДАННЫХ

3.1 Условия проведения эксперимента

В рамках экспериментальной проверки работы имитатора было сгенерировано и поставлено в очередь диспетчера $N = <\text{N}>$ заявок. Все четыре вычислительных узла кластера на момент начала эксперимента находились в состоянии готовности (idle).

3.2 Результаты постановки заявок в очередь

По завершении выполнения всех сгенерированных заявок были собраны итоговые статистические данные, фрагмент которых приведён в таблице 4.

Таблица 4: Сравнение плановых и фактических характеристик выполнения заявок (фрагмент)

JobID	$T_{\text{заявки}}$, с	$T_{\text{факт}}$, с	NNodes	State
<...>	<...>	<...>	<...>	COMPLETED

3.3 Анализ отклонений «план – факт»

Сопоставление запрошенного пользователем времени выполнения $T_{\text{заявки}}$ и фактического времени выполнения $T_{\text{факт}}$ по результатам эксперимента позволило вычислить выборочные значения среднего и стандартного отклонения логарифма отношения τ для синтетических данных и сравнить их с параметрами μ и σ, полученными на этапе анализа исходного набора acct_0921-0923 (подраздел 2.4).

Полученные по результатам эксперимента значения $\hat{\mu}$ = <значение> и $\hat{\sigma}$ = <значение> <близки / отличаются от> соответствующих параметров исходной выборки, что подтверждает (либо позволяет уточнить) адекватность выбранной статистической модели генерации пользовательских заявок.

3.4 Выводы по результатам экспериментов

Проведённый эксперимент показал, что разработанный имитатор пользовательских заявок обеспечивает корректную автоматическую постановку заданий в очередь диспетчера SLURM, а статистические характеристики сгенерированной нагрузки воспроизводят характер распределения времени выполнения и ошибки его оценки, наблюдаемый в исходном наборе учётных данных. Это подтверждает применимость предложенного подхода к моделированию пользовательской нагрузки на вычислительный кластер при отсутствии доступа к данным реальной эксплуатационной системы в реальном масштабе времени.

Заключение

В процессе выполнения НИР были изучены принципы организации виртуальной вычислительной среды, имитирующей ресурсы вычислительного кластера, а также технологии управления заданиями средствами системы SLURM на базе дистрибутива Linux Fedora Server в среде виртуа-

лизации Oracle VM VirtualBox. В результате работы были выполнены поставленные задачи:

1. Установлена и настроена среда виртуализации Oracle VM VirtualBox.
2. Создано пять виртуальных машин: один управляющий узел и четыре вычислительных узла кластера.
3. Установлен дистрибутив Linux Fedora Server (архитектура aarch64) на все виртуальные машины в режиме интерфейса командной строки.
4. Выявлена и устранена проблема совместимости эмулируемого контроллера накопителей с архитектурой ARM хост-системы путём перехода на контроллер VirtIO SCSI.
5. Настроена виртуальная локальная сеть между узлами кластера с разрешением имён посредством файла /etc/hosts.
6. Настроено подключение по протоколу SSH с авторизацией по криптографическим ключам между управляющим и вычислительными узлами.
7. Установлена и настроена служба аутентификации munge, а также диспетчер заданий SLURM (slurmctld на управляющем узле и slurmd на вычислительных узлах).
8. Выполнено клонирование настроенного вычислительного узла для создания оставшихся узлов кластера с последующей индивидуализацией их сетевых параметров.
9. Проведён анализ набора учётных данных acct_0921-0923 по заданным атрибутам варианта (UID 50430, JobName “run_bwa.sh”), вычислены границы диапазона фактического времени выполнения задач, а также параметры μ и σ логарифма ошибки оценки времени выполнения.
10. Разработан имитатор пользовательских заявок, генерирующий фактическое время выполнения задачи по нормальному распределению в вычисленном диапазоне и пользовательскую оценку времени выполнения по логнормальному распределению с вычисленными параметрами, с автоматической постановкой сформированных заявок в очередь диспетчера SLURM.
11. Реализован сбор и сохранение в файл статистики поставленных в очередь заявок для последующего сравнительного анализа плановых и

фактических характеристик их выполнения.

Таким образом, в ходе выполнения НИР развёрнута работоспособная виртуальная модель вычислительного кластера, на базе которой реализован и экспериментально проверен имитатор пользовательской нагрузки, статистические характеристики которого согласуются с параметрами, полученными в результате анализа реального набора учётных данных. Полученные результаты могут быть использованы в качестве основы для дальнейшего расширения функциональности диспетчера и более детального анализа эффективности алгоритмов планирования заданий.

Список литературы

Приложение А. Исходный код имитатора